

# SARVAM Assignment

Name - Paarth Saxena

Email - paarthsaxena2005@gmail.com

Number - +91 9319277334

# **PART - A**

## **Question 1**

### 1. Android Process Kill During Inference

On the Snapdragon device, the model process is killed by the Android OS memory manager mid-inference. The client is waiting for a response.

Design the **full recovery path**, including:

- What the **local agent** does in the first **500ms** after the kill
- How it signals the **gateway**
- How the gateway handles the **in-flight request**
- What state must be **checkpointed** to make recovery fast

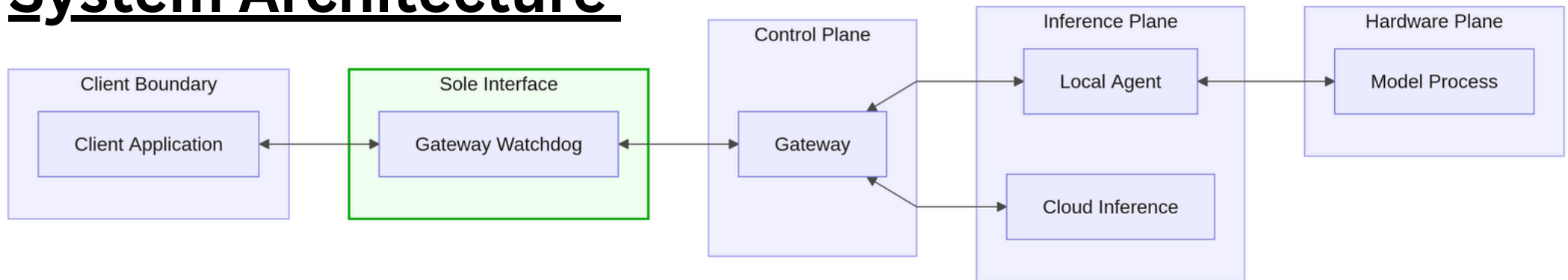
You must provide:

- Exact timeout values
- Retry behaviour
- Recovery sequencing
- Justification for all timeout and state-management decisions

# Assumptions

- **Process Isolation**— The AI model inference runs in a separate isolated process, so killing it doesn't terminate the whole application and recovery can happen without restarting the complete app.
- **Streaming Token Generation** — Generated tokens are streamed incrementally, which reduces perceived latency, enables partial response persistence, and allows watchdog systems to detect stalled generation.
- **Fixed Inference Parameters** — Inference parameters (temperature, top-k, top-p, sampling settings) are fixed per request and re-transmitted on cloud handoff. This reduces the parameter space that needs to be re-transmitted and avoids inconsistent continuation behavior between local and cloud inference
- **Warm-Up Profiling Phase** - Baseline values for p99 local inference latency, p95 cloud latency, and p90 network RTT are pre-computed during the development phase on representative Snapdragon hardware.

# System Architecture



- **Local Agent**— Spawns/manages model process, streams tokens, monitors health, handles cleanup/restart, maintains inference state. Uses protobuf for low-overhead communication with the Gateway.
- **Gateway** — Orchestration layer that receives requests, routes locally/remotely, monitors failures, manages retries, streams responses via Watchdog, records metrics, handles fallback logic.
- **Gateway Watchdog** - Sole communication interface between client and inference pipeline. Monitors token streaming and acts as timeout authority for the full inference lifecycle. Client has no direct connection to Gateway.
- **Model Process** — Emits continuous **heartbeats** to Local Agent for health monitoring and deadlock detection. Notifies Local Agent before termination for controlled teardown.

# Timeout and heartbeat Design

- **Heartbeat Frequency: 100 ms**

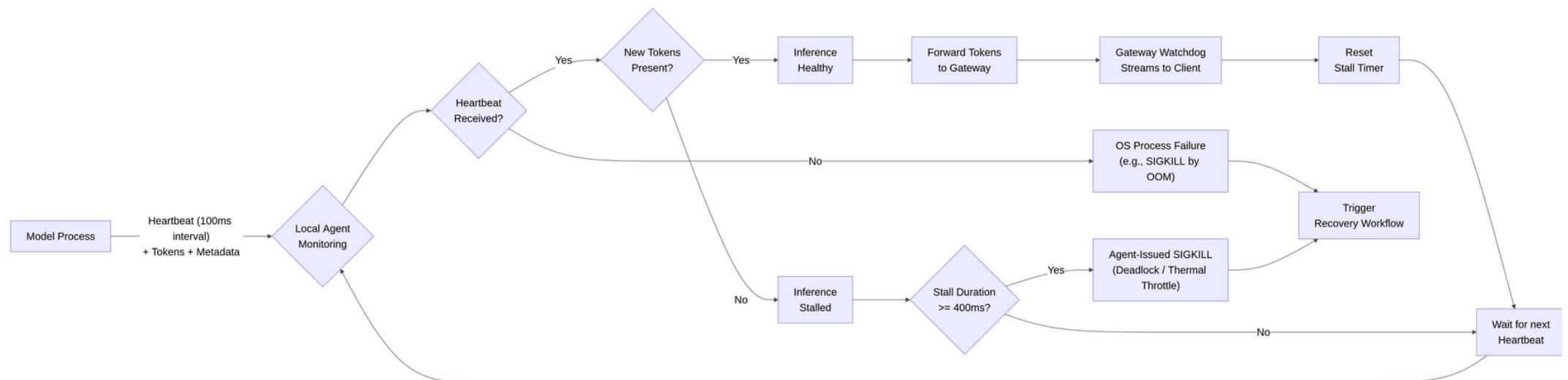
- generally inter process communication happens within microseconds or few ms so 100ms is decent interval having high frequency without adding unnecessary overhead.
- Model Process sends heartbeat to local agent to tell that it is alive
- A process can still be alive while inference is stuck,
  - we send the Token progress alongside each heartbeat
  - this token progress is forwarded to the client side application to also do streaming responses

- **Inference Timeout Threshold: 400 ms**

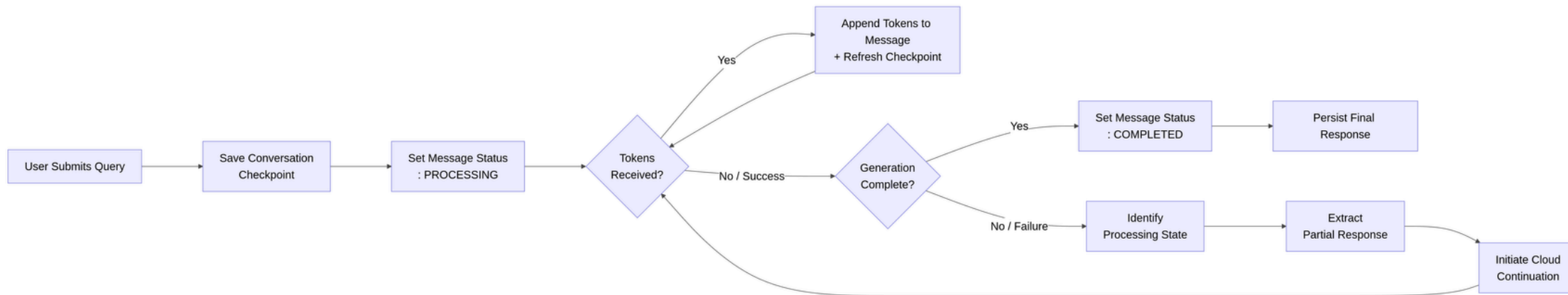
- Local inference generally operates within microseconds to low milliseconds.
- A threshold below 300 ms may create false positives on low-end hardware.
- More than 400 ms strongly indicates instability or deadlock.

- **Recovery Actions (400ms - 500ms)**

- Forcefully terminate model process using **SIGKILL**
- Perform synchronous memory cleanup, we want to do cleanup before emitting event for the Gateway, removes memory like
  - IPC communication pipes
  - Stack allocations
  - Temporary buffers
  - Runtime handles
- Notify Gateway with cleanup confirmation flag and partial token count, uses protobuf for speed and efficiency the also works great in low end devices too making it less bulky and hence making it faster.



# State Management and Checkpoints



- **Initial Query Submission**

- Checkpoint stored Immediately after the user clicks "Send" but before inference begins.
- This ensure the latest user query is never lost. Even if the application crashes or the inference engine fails to start, the system can identify the "unfinished" state upon restart and resume the workflow.

- **Status Transition to Processing**

- Concurrent with the initial checkpoint.
- This establish a unique "active" message. The system enforces that only one message can be in a Processing state at a time, preventing race conditions and defining the exact target for partial token updates.

- **Incremental Token Updates (Streaming)**

- Every time a new chunk of tokens arrives from the Local Agent or Cloud.
- **Minimize Lost Work:** If a crash occurs mid-generation, only the tokens from the last few milliseconds are lost.
- **Partial Response Continuity:** It preserves the "fixed prefix" needed to construct the continuation prompt for cloud fallback, ensuring the user doesn't see a contradictory or repeating response.

- **Continuous Checkpoint Refreshing**

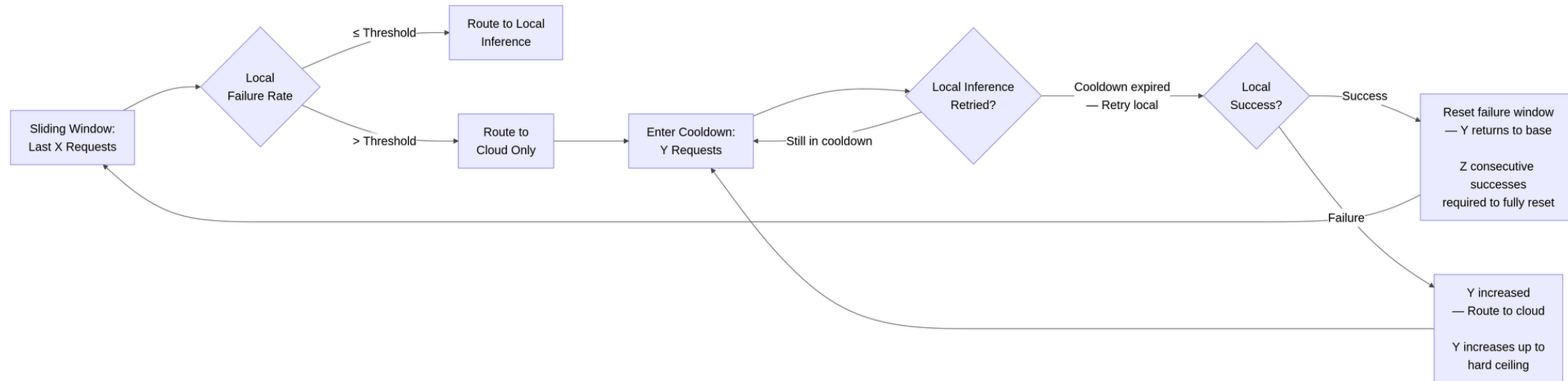
- Every time the message content is updated with new tokens.
- This keep the persistent storage in sync with the in-memory stream. This makes the recovery "crash-proof"—if the OS kills the app, the state on disk represents the most recent progress.

- **Final Status Transition to Completed**

- Only after the "End of Sentence" token is received and the stream is closed.
- **Conversation Context:** Marks the message as a permanent part of the history to be used as context for future prompts.
- **Cleanup:** Signal to the Gateway and UI that the generation cycle is finished and the system can return to an Idle state.

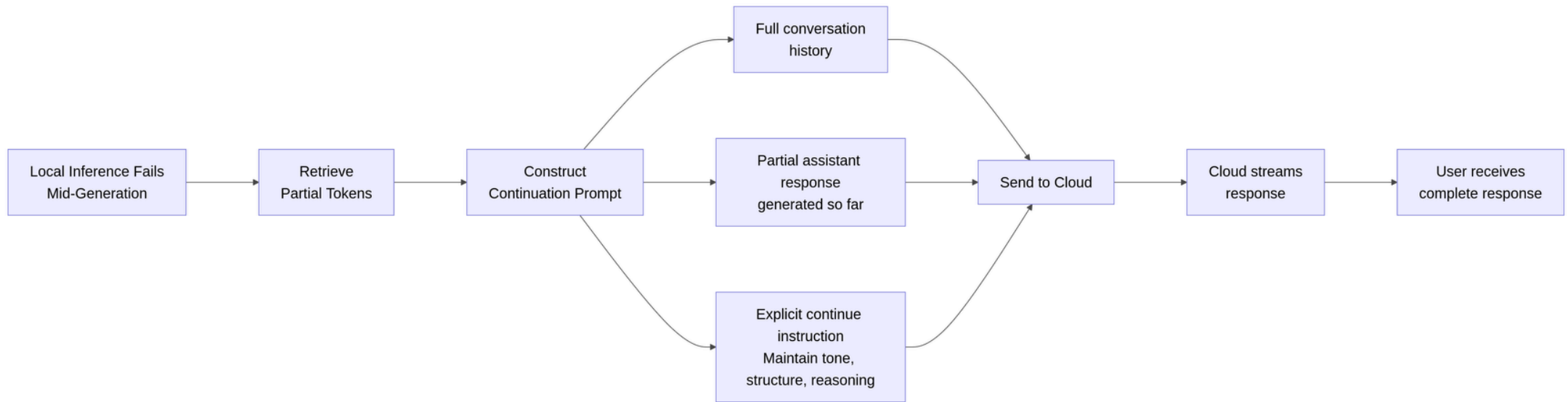
# Fail Safe Mechanism, Recovery and Retry.

- **Backoff CoolDown Strategy.**



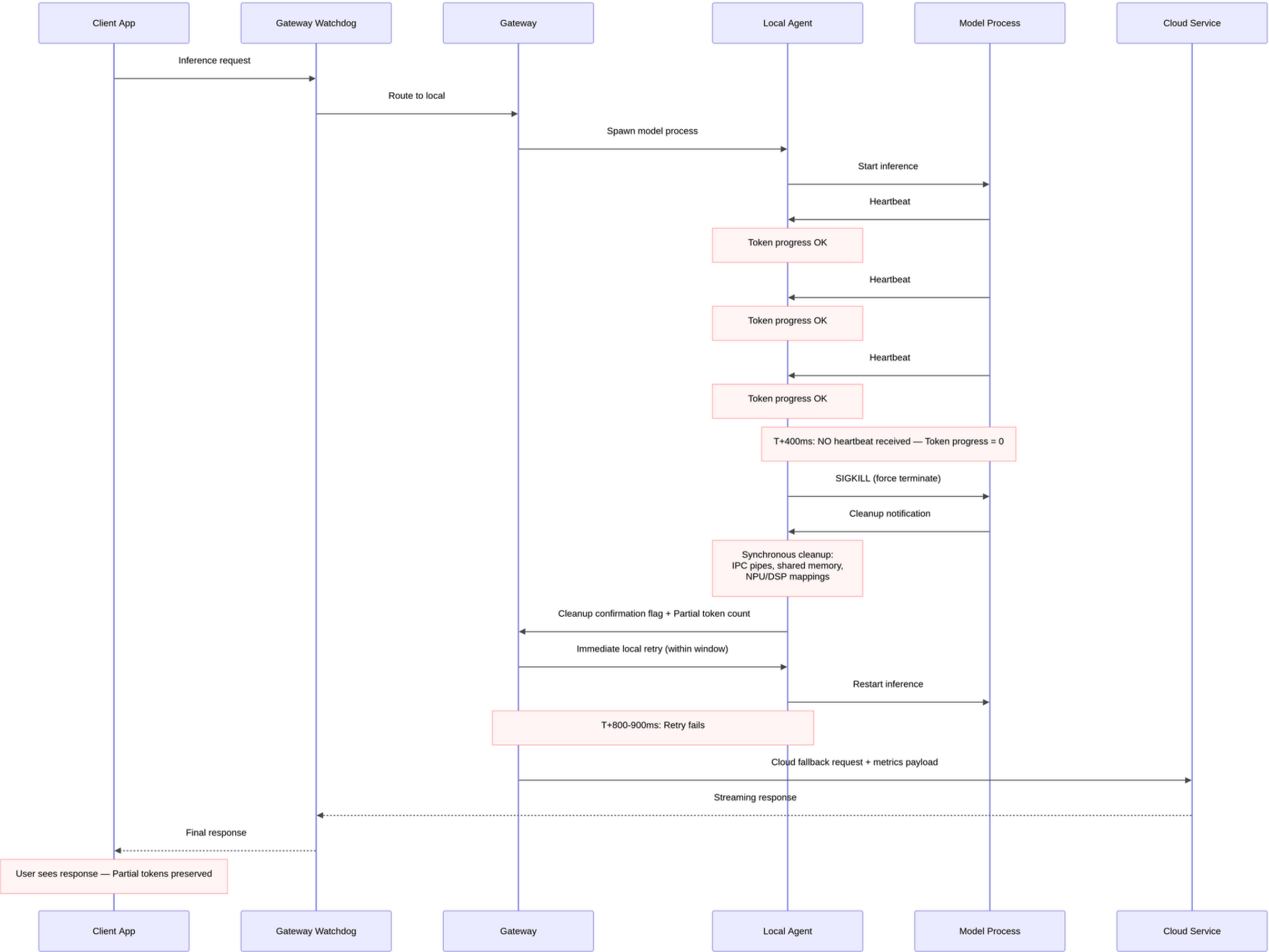
- A traffic-routing mechanism that monitors a sliding window of local failures and temporarily redirects all requests to the cloud if stability thresholds are breached
- It helps the local system recover by preventing a loop where repeated local attempts worsen the performance and even create a dead end
- the cooldown mechanism's duration scales dynamically based on failure frequency, persistent and but ensures that the user side performance is not compromised and the device gets more recovery time if the underlying hardware stress remains high.
- A hard ceiling on the cooldown ensures the device is never permanently locked out of local inference, allowing it to reclaim on device capabilities once it stabilizes

## • **Partial Response Continuation Flow**



- A recovery mechanism that captures the exact sequence of tokens generated locally before a failure and uses them as a "fixed prefix" for the Cloud model to continue from.
- It is useful because it prevents the model from generating a response that contradicts or diverges from the text the user has already seen on their screen.
- By continuing instead of regenerating from scratch, the system respects the user's time and avoids the computational cost of re-processing tokens that were already successfully delivered.
- The flow works by bundling the full conversation history, the partial response, and an explicit "continue" instruction into a single payload, ensuring a seamless transition that maintains the original tone and reasoning thread.

# End to End Recovery Sequence





# Cleanup Responsibilities

- **Local Agent**
  - Issues SIGKILL & Executes Synchronous Cleanup
  - Ensures IPC pipes, Shared Memory, and NPU/DSP mappings are released atomically before notifying Gateway.
- **Model Process**
  - Emits Heartbeats & Cleanup Notification
  - Provides liveness signal; allows the Agent to synchronize state before forceful termination.
- **Gateway**
  - Evaluates Cleanup Confirmation Flag
  - Guarantees that retries only occur when the hardware and memory state are known to be clean.

# Question 2

## 2. Firmware Update Breaking NPU Driver Compatibility

A firmware update on **200 HP laptops** changes the NPU driver. Your model binary now crashes on load on those devices but works on all others.

You have:

- No physical access
- Only remote log access

Describe exactly how you would:

(a) Detection

Detect which devices are affected using only:

- Logs
- Heartbeat telemetry
- Crash metadata

(b) Root Cause Isolation

Isolate the root cause without reproducing the issue locally.

(c) Mitigation

Ship a mitigation **without a full model recompile**.

(d) Prevention

Prevent this class of failure in future deployments.

Your answer should include:

- Rollout strategy
- Driver/version fingerprinting
- Canary deployment logic
- Telemetry improvements
- Fallback execution paths

# a) Detection

- **Structured Observability Pipeline**

- need to make the logs and metrics queryable rather than pure text
- we can use prometheus and grafana and hence make it queryable so that it is easy for us to club and detect issues collectively

- **Log Enrichment**

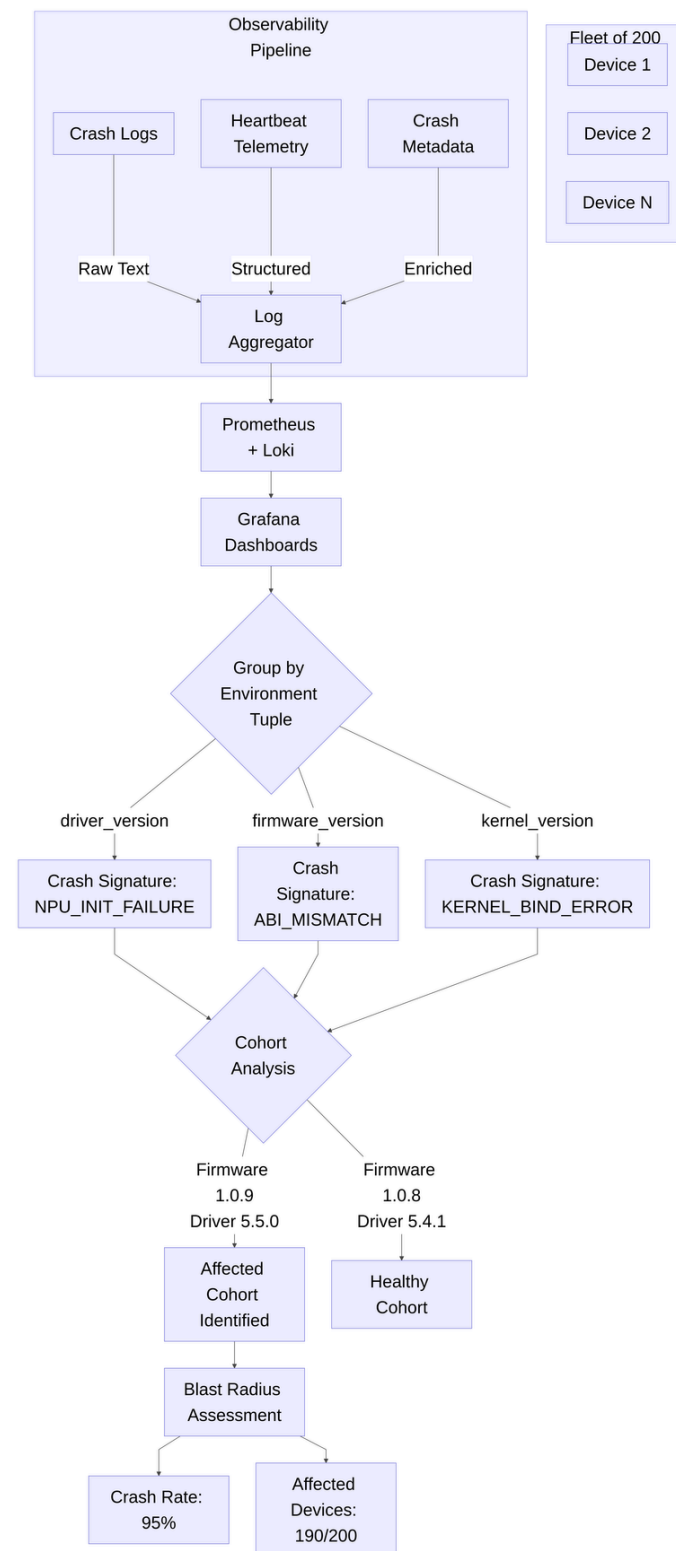
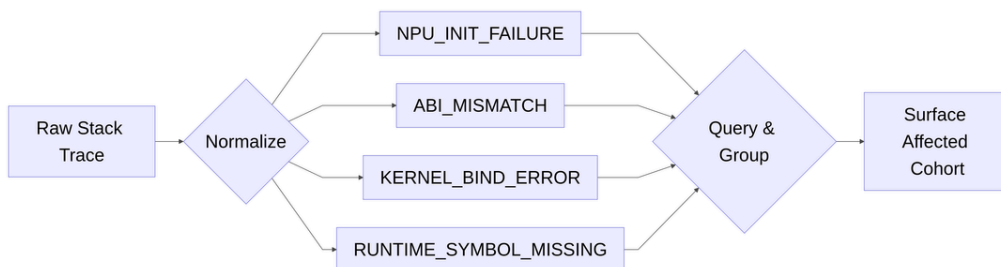
- We should enrich the logs with more metadata like machine specs and system metrics.
- This would help us understand and diagnose the problem better

- **Blast Radius and Cohort**

- We should check the blast radius for a particular issue, i.e. how many devices are affected and what is the magnitude of this.
- It also helps us create cohorts and groups to target and solve for them properly.

- **Crash Signature Normalization**

- Raw stack traces are noisy and vary across devices. Normalizing them into identifiers like NPU\_INIT\_FAILURE or ABI\_MISMATCH allows for high-level SQL-like querying across a fleet of thousands.
- It helps us to group problems together and even understand the different conditions under which we see the same issues
- It helps distinguish between software bugs and environmental issues , compatibility issues etc.
- This is really helpful when stored with the error stack cause then it quickly helps to identify and diagnose the problem itself



# **b) Root Cause Isolation**

- **Build and Validate Hypotheses Using System Metrics**

- Root cause isolation should be hypothesis-driven. Rather than jumping to conclusions, form hypotheses based on the crash stage and validate or invalidate each one using system metrics and logs.
- **Hypothesis 1 : kernel Compatibility issue**
  - The process fails before it can even request memory or bind to the kernel.
  - System metrics show memory allocation remains at 0 and the process lifetime is extremely short (< 10ms).
- **Hypothesis 2: ABI / Symbol Mismatch**
  - Initialization completes successfully, but the process crashes immediately upon the first execution call or hardware interaction.
  - Memory loads to the expected baseline, but a crash signature is generated at the exact moment of the first CPU/NPU spike.
- **Hypothesis 3: Memory Leak / Stall (Zombie State)**
  - The process is alive and consuming resources but has entered a deadlock or infinite loop.
  - Memory climbs steadily (linear growth) without plateauing, while CPU utilization for the process remains at 0.

- **Reconstruct Execution Timeline**

- Using logs, telemetry, and crash metadata together, reconstruct the exact sequence of events:
  - model deserialize
  - graph compile
  - kernel bind
  - memory allocation
  - warmup inference
  - crash
- The exact stage where execution stops significantly narrows the root cause candidates and helps select which hypothesis to investigate first.

- **Differential Analysis — Working vs Failing Devices**

- Compare affected devices directly against unaffected devices with similar hardware configurations
- This approach is especially useful for validating hypotheses. If a hypothesis is correct, the pattern should be consistent across all failing devices and absent in working devices with similar hardware. If working devices with the same hardware show no issue, that further isolates the cause to the driver or firmware delta.

# c) Mitigation

- **Halt Rollout**

- Disable update distribution to HP cohort immediately
- Prevent the "Blast Radius" from expanding.

- **Feature Flag Fallback**

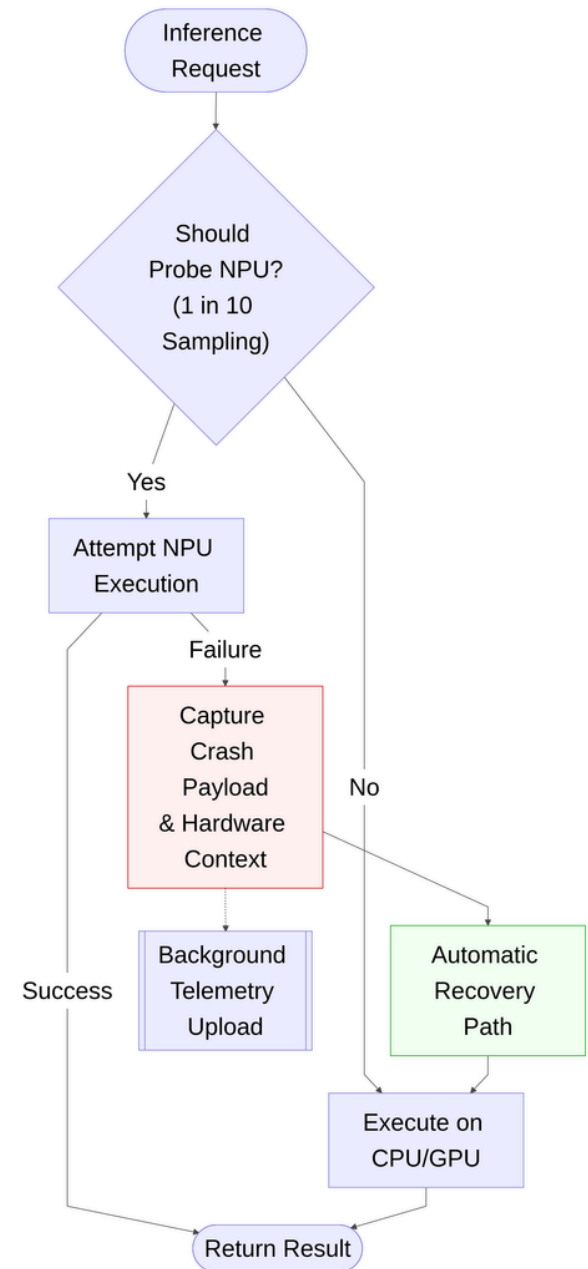
- Remote toggle: Force CPU/GPU execution for `Driver 5.5.0`
- Restore service functionality with degraded performance.

- **Probabilistic Probing**

- Route 1 in 10 requests to NPU (with auto-fallback to cpu and gpu utilization).
  - 1 in 10 is an arbitrary number and will be dependent on a lot of variables in the actual environment.
- Collect diagnostics on live hardware without user impact.
- This helps in collecting more data and logs without interfering with user experience

- **Verbose Logging**

- Activate debug-level telemetry for the affected cohort.
- Increase the logs quality for better diagnoses.



# d) Prevention

- **Cohort-Aware Canary Deployments**

- Segmented Rollouts: Group the fleet into "Environment Cohorts" based on specific hardware, driver, and firmware versions rather than using random sampling.
- Strategic Sampling: For large cohorts, use staged rollouts (1% → 5% → 20% etc.); for small, high-risk cohorts (like the HP 1.0.9 group), deploy as a single test block to gather compatibility data rapidly.
- Data-Driven Gates: Monitor crash rates and heartbeat stability specifically within the canary cohort before allowing the update to proceed to the broader

- **Live Compatibility Matrix**

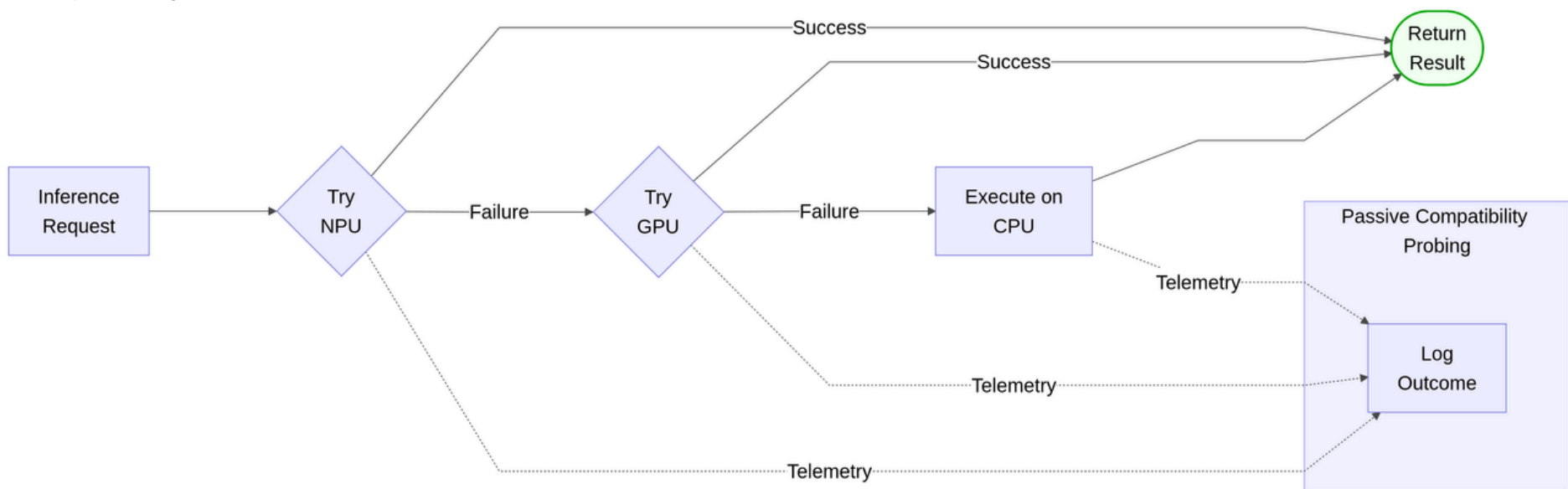
- Environment Mapping: A Mapping between the versioning of the system specs and the code version so that we have a way to compatibility between software and the actual hardware.

- **Runtime Fallback Execution Path (Standard Architecture)**

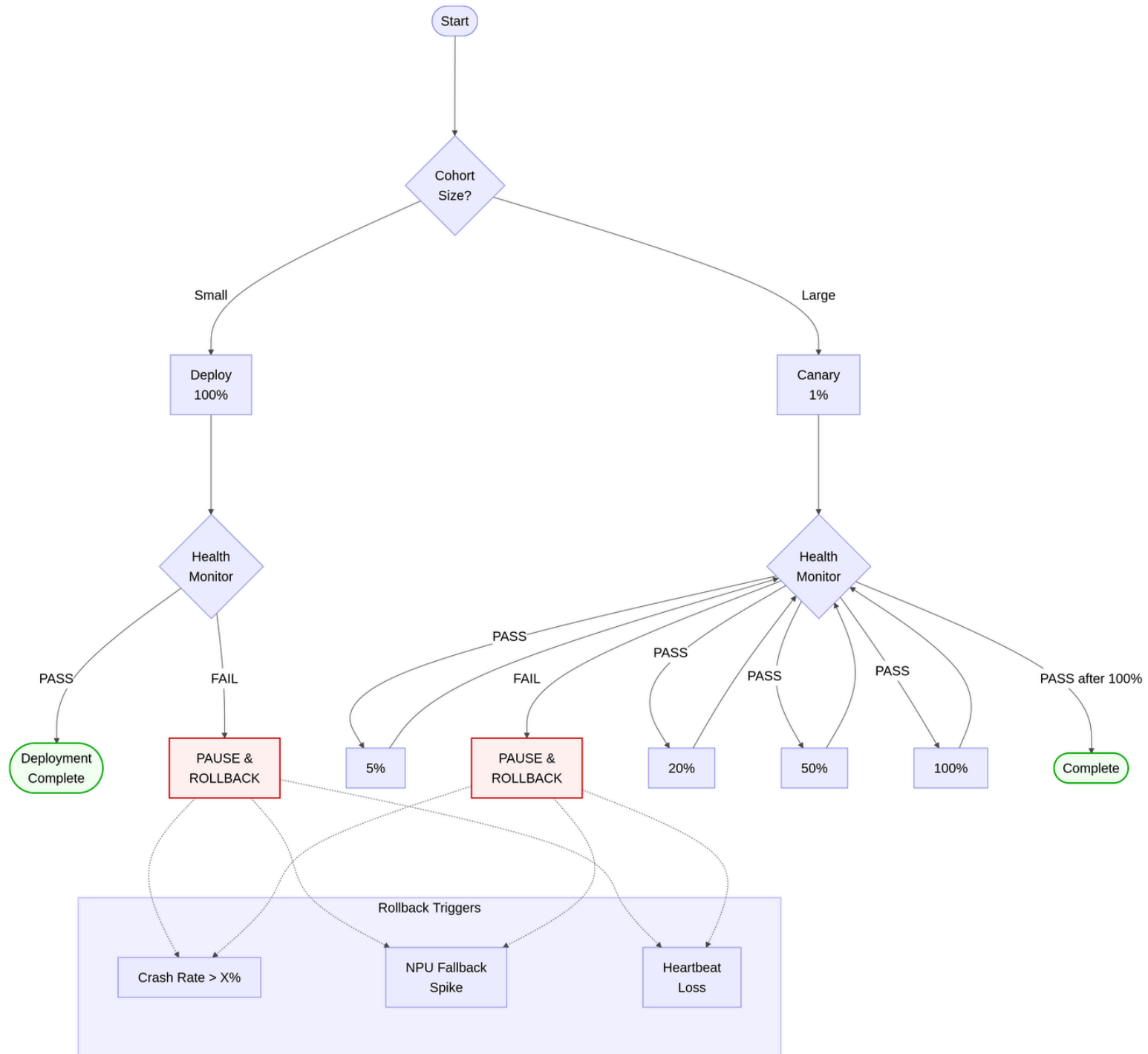
- NPU → GPU → CPU Priority: Shift fallback from a "emergency mitigation" to a core architectural standard. The system should natively attempt the NPU and transparently fall back to more stable providers (GPU/CPU) on any hardware failure.
- Passive Compatibility Probing: This architecture help us to test and validate the new changes we make without compromising the user experience.

- **Automatic Rollback Gates**

- Implement automatic deployment halts that fire if the crash rate exceeds a specific threshold or if "Fallback Activation" spikes unexpectedly in a new cohort.



# Canary deployment methods



# Question 3

## 3. INT4 Quantisation Failures on Snapdragon

The model uses **INT4 quantisation** to fit within **4GB RAM**, but produces incorrect outputs (hallucinations) for a percentage of requests specifically on **Snapdragon Series X** devices. The issue is not reproducible on other hardware.

Provide your **top 3 hypotheses** for the root cause.

Your answer must be specific about:

- Quantisation behaviour
- Hardware execution differences
- Numerical instability
- Kernel/runtime implementation differences

For each hypothesis, include:

- The exact diagnostic probe you would add
- The minimal code change needed to validate or falsify the hypothesis
- The expected signal/results if the hypothesis is correct



# Hypothesis 1

- **Hypothesis**

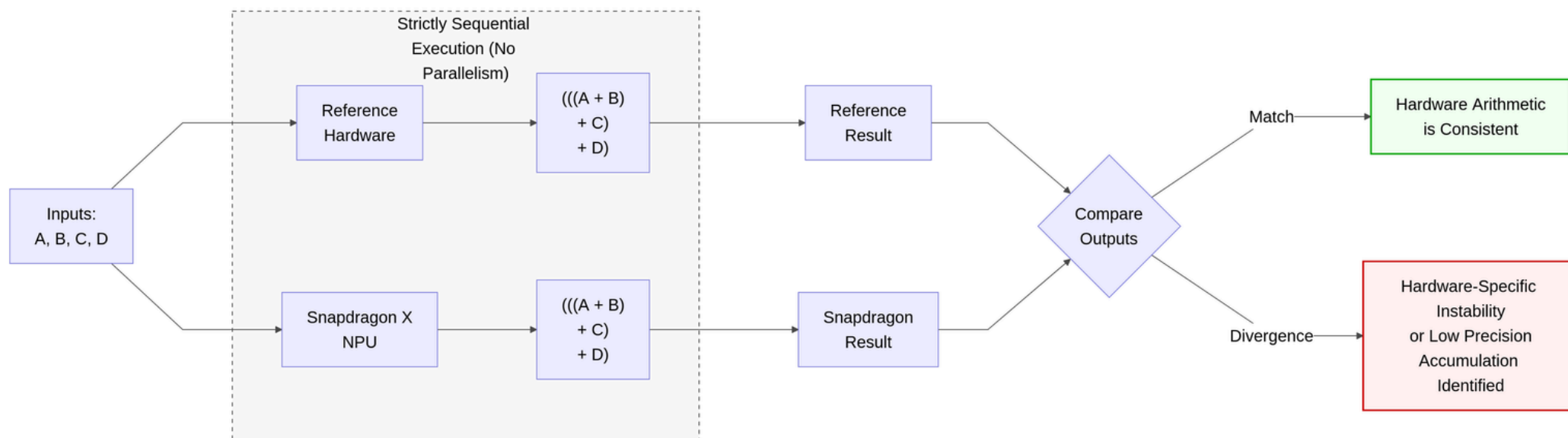
- Snapdragon X NPUs may use hardware-specific low precision execution paths (e.g., FP16 accumulation, approximate math, vectorized optimizations) that behave differently.
- INT4 has extremely low precision headroom; tiny numerical deviations amplify deviation from the actual response especially in LLM as the number of parallel calculations increase.

- **Diagnostic Probe**

- After each transformer layer's matrix multiplication on the NPU, dequantize the output tensor to FP32 and log the maximum absolute deviation against the same operation run on CPU reference hardware. Track which specific layer the divergence first appears at and whether it grows through subsequent layers.

- **Expected Signal**

- Initially the values would not be much different but as the layers go on and the calculations start stacking up we would see quite some divergence between the results



# Minimal Code Change:

```
1 import torch
2 import logging
3
4 log = logging.getLogger(__name__)
5
6 # Stores FP32 reference activations captured from CPU execution
7 CPU_REFERENCE_ACTIVATIONS = {}
8
9 def probe_layer_activation(tensor_out, layer_id, device_type):
10     """
11     Compare Snapdragon INT4 activations against CPU FP32 reference.
12
13     Args:
14         tensor_out: Quantized tensor output from current layer
15         layer_id: Unique identifier for layer
16         device_type: 'cpu' or 'snapdragon_x'
17     """
18     # Convert quantized tensor back to FP32 for comparison
19     fp32_out = tensor_out.dequantize().to(torch.float32)
20
21     if device_type == "snapdragon_x":
22         # Retrieve CPU reference activation
23         ref = CPU_REFERENCE_ACTIVATIONS.get(layer_id)
24         if ref is not None:
25             # Compute activation divergence
26             max_delta = torch.max(torch.abs(fp32_out - ref)).item()
27             mean_delta = torch.mean(
28                 torch.abs(fp32_out - ref)
29             ).item()
30             # Structured logging
31             log.info({
32                 "event": "activation_probe",
33                 "layer_id": layer_id,
34                 "max_delta": max_delta,
35                 "mean_delta": mean_delta,
36                 "device": device_type
37             })
38             # Detect abnormal divergence
39             if max_delta > 0.05:
40                 log.warning(
41                     f"Divergence detected at layer {layer_id}"
42                 )
43         else:
44             # CPU execution path:
45             # Store reference activations
46             CPU_REFERENCE_ACTIVATIONS[layer_id] = fp32_out.clone()
```

# Hypothesis 2

- **Hypothesis**

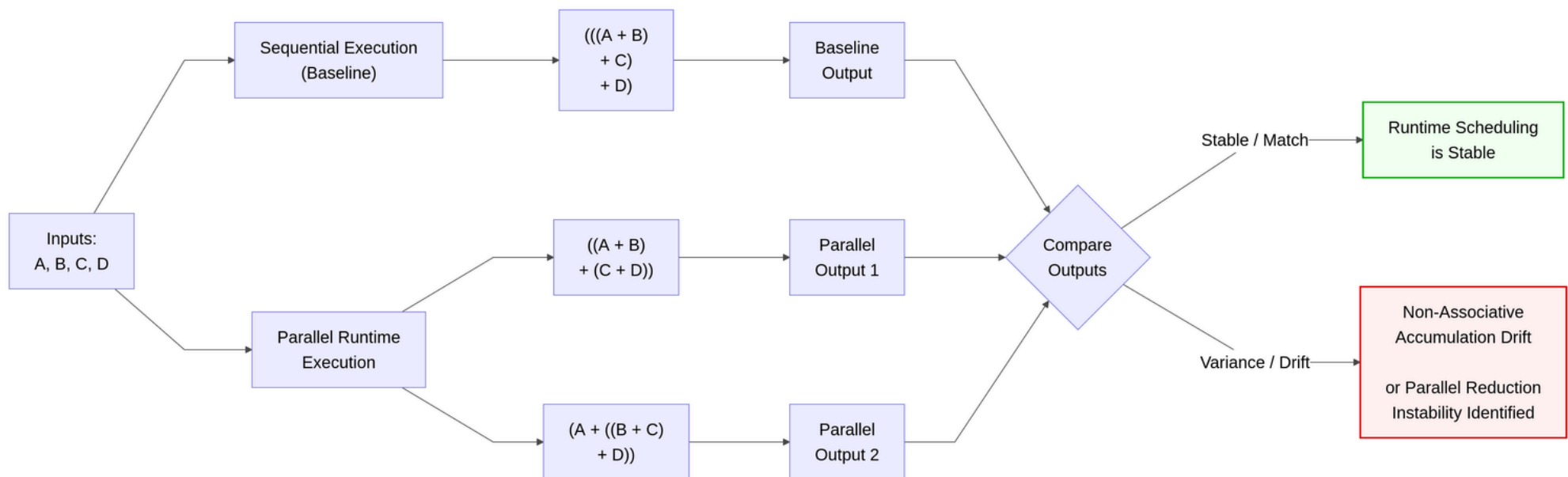
- The Snapdragon runtime may execute operations in a different order due to parallel reductions, thread scheduling, or async execution.
- Floating-point arithmetic is non-associative  $(a+b)+c \neq a+(b+c)$ . Changing the execution order can produce different intermediate activations and deviation from expected result

- **Diagnostic Probe**

- Force the Snapdragon runtime to single-threaded execution by setting thread count to 1, then run identical inference twice – once with default parallel execution, once forced sequential. Log the output delta between the two runs. Also log the delta between single-threaded Snapdragon output and CPU reference output

- **Expected Signal**

- The cpu compared to sequential would near identical result whereas the spu vs parallel there would be divergence and hence this would confirm that the order of operation in cpu is the cause of this issue.



# Minimal Code Change:

```
1 import os
2 import torch
3 import logging
4
5 log = logging.getLogger(__name__)
6
7
8 def run_determinism_probe(input_tensor, model, device_type):
9
10     if device_type != "snapdragon_x":
11         return
12
13     os.environ["QNN_NUM_THREADS"] = "4"
14     output_parallel = model.run_inference(input_tensor)
15
16     os.environ["QNN_NUM_THREADS"] = "1"
17     output_sequential = model.run_inference(input_tensor)
18
19     output_cpu = model.run_inference_cpu(input_tensor)
20
21     delta_parallel_vs_sequential = torch.max(
22         torch.abs(output_parallel - output_sequential)
23     ).item()
24
25     delta_sequential_vs_cpu = torch.max(
26         torch.abs(output_sequential - output_cpu)
27     ).item()
28
29     log.info({
30         "event": "determinism_probe",
31         "parallel_vs_sequential_delta":
32             delta_parallel_vs_sequential,
33
34         "sequential_vs_cpu_delta":
35             delta_sequential_vs_cpu,
36
37         "device": device_type
38     })
39
40     if delta_parallel_vs_sequential > 0.01:
41         log.warning("Non-deterministic parallel execution detected")
42
43     if delta_sequential_vs_cpu > 0.05:
44         log.warning("Sequential Snapdragon output diverges from CPU")
```

# Hypothesis 3

- Hypothesis

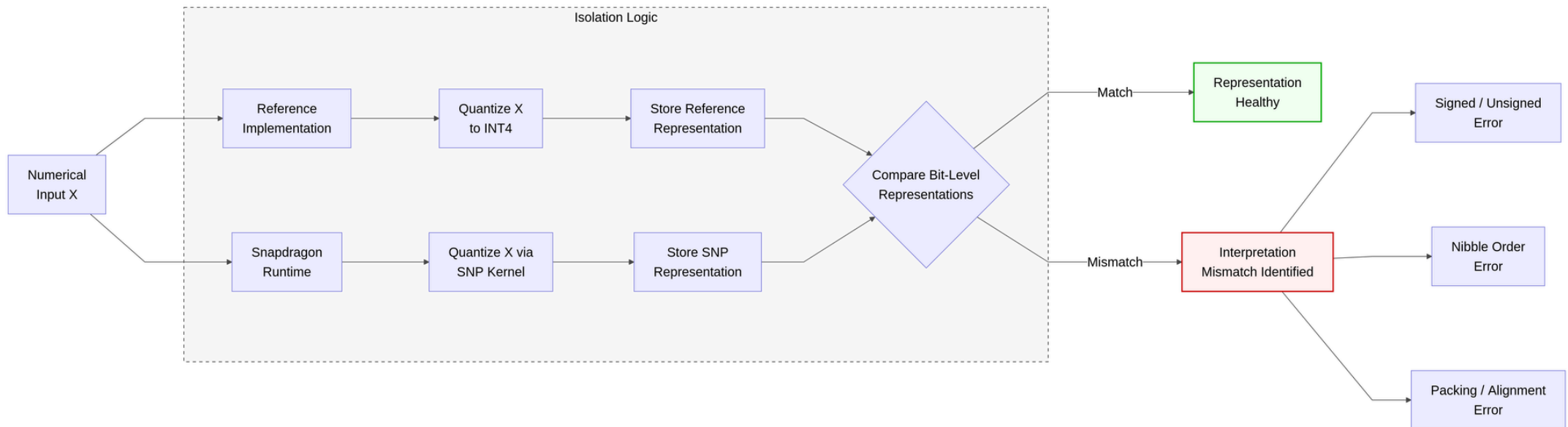
- Snapdragon-specific INT4 kernels may incorrectly interpret quantized tensor representations.

- Diagnostic Probe

- Immediately after the Snapdragon kernel performs INT4 quantization, capture the raw byte output and compare it bit-for-bit against a CPU reference quantization of the same input. Log the first mismatched byte from both sides in binary to identify whether it is a nibble order, sign bit, or alignment issue.

- Expected Signal

- the binary representation differences in bits would indicate that it is a representation issue itself.



# Minimal Code Change:

```
1 import torch
2 import logging
3
4 log = logging.getLogger(__name__)
5 logging.basicConfig(level=logging.INFO)
6
7 def pack_nibbles_reference(x_int: torch.Tensor) -> bytes:
8     """Pack INT4 values two per byte, low nibble first (reference spec)"""
9     x_flat = x_int.flatten().tolist()
10    packed = bytearray()
11    for i in range(0, len(x_flat), 2):
12        low = int(x_flat[i]) & 0xF
13        high = int(x_flat[i+1]) & 0xF if i+1 < len(x_flat) else 0
14        packed.append((high << 4) | low)
15    return bytes(packed)
16
17 def probe_int4_representation(input_tensor, scale, zero_point, snpe_context):
18     # Reference CPU quantization
19     x_int = torch.clamp(
20         torch.round(input_tensor / scale) + zero_point, -8, 7
21     ).to(torch.int8)
22     ref_packed = pack_nibbles_reference(x_int)
23
24     # Snapdragon kernel quantization
25     snp_packed = snpe_context.quantize_int4(input_tensor) # raw bytes
26
27     mismatch_count = sum(
28         1 for a, b in zip(ref_packed, snp_packed) if a != b
29     )
30
31     log.info({
32         "event": "int4_representation_probe",
33         "total_bytes": len(ref_packed),
34         "mismatch_count": mismatch_count,
35         "mismatch_rate": mismatch_count / len(ref_packed)
36     })
37
38     if mismatch_count > 0:
39         # Log first mismatch in binary for pattern identification
40         for i, (r, s) in enumerate(zip(ref_packed, snp_packed)):
41             if r != s:
42                 log.warning({
43                     "first_mismatch_index": i,
44                     "ref_byte": f"{r:08b}", # e.g. 00110101
45                     "snp_byte": f"{s:08b}", # e.g. 01010011 -> nibble swap
46                 })
47                 break
48
```

# ***PART - B***

- The complete codebase, benchmark outputs, metrics, and test reports are available in the GitHub repository.

## **GitHub Repository**

- **Steps to Run the Application**

- **Edit .env** — set SARVAM\_API\_KEY to your real key (otherwise server returns arbitrary values)
- **Run docker compose build**
- **Run docker compose up -d**
  - **Test health:** `curl http://localhost:8000/api/v1/tickets/health`
  - **Test Ticket parse:** `curl -X POST http://localhost:8000/api/v1/tickets/parse \ -H "Content-Type: application/json" \ -d '{"tickets": ["Cannot login to my account, it keeps saying wrong password", "Server is down at our office"]}'`
- **View logs:** `docker compose logs -f ticket-pipeline`
- **Stop:** `docker compose down`